

PARTIE 1 — PARCOURS ENSEIGNANT : DÉCOUVERTE DES FORMATIONS

1. Catalogue de formations enseignant

✗ ABSENT

Vérification	Résultat	Preuve
Modèle dédié "Formation/Training"	Inexistant	Aucun modèle Formation, Training, TeacherTrain (lignes), models_lms.py (410 lignes), mode
Endpoint backend /formations ou /trainings	Inexistant	Aucun match pour `training
Champ course_type ou audience sur le modèle Course	Inexistant	Course (models.py:427-508) a visibility, ov audience enseignant/élève
Page frontend "My Learning"	Existe mais cassée	frontend/src/features/teacher/pages/MyLe formation). Ligne 68 appelle POST /api/ac inexistant dans academy.py
Route /dashboard/teacher/learning	Enregistrée	App.tsx:180 — charge MyLearning avec F
Filtrage par matière/type/durée	Inexistant	Aucun endpoint de catalogue ne supporte
Formations gratuites vs payantes	Inexistant	Le modèle Course a des champs price/pri pour enseignant"

Conclusion : Le frontend a une coquille UI (MyLearning.tsx) avec des tests unitaires (test/MyLearning.test.tsx, 208 lignes) mais aucun backend ne supporte cette fonctionnalité. Pas de modèle, pas d'endpoint, pas de logique métier.

2. Restriction selon l'état de l'enseignant (essai / vérifié / indépendant)

✗ ABSENT

Vérification	Résultat	Preuve
Rôles enseignant distincts	Partiel	Les rôles existent : teacher, admin_school, pedagogical_lead, independent_teacher (Us dans models.py:97-104) mais aucun endpoint ne filtre les formations p enseignant
Accès différent par état	Inexistant	Pas de concept de "formation réservée aux enseignants vérifiés"
Wallet essai pour achats enseignant	Inexistant	Le wallet essai (trial pool dans WalletTransaction) est accordé à l'insc (auth.py:115-121) mais il n'y a pas de mécanisme d'achat de formatio enseignant

3. Parcours d'achat d'une formation

✗ ABSENT

Vérification	Résultat	Preuve
Sélection d'une formation payante	Inexistant	Pas de catalogue de formations
Affichage du prix	Inexistant	MyLearning.tsx:195-199 affiche price_tokens et price_dt mais les données sont mockées (pas d'API)
Fournisseur de paiement câblé	Stripe uniquement	Stripe est le seul fournisseur (payments/stripe.py). Aucun Paymee/Konnect/Flouci. L'achat de cours (courses.py:309-366) ne déduit aucun solde — il crée un enregistrement CoursePurchase sans transaction financière

Vérification	Résultat	Preuve
Confirmation post-paiement	Inexistant	Pas de flux d'achat fonctionnel pour les formations
Cas d'échec de paiement	Inexistant	Le endpoint <code>purchase_course</code> ne vérifie pas le solde (<code>courses.py:309-366</code>)

Note : L'achat de cours (`courses.py:338`) crée un `CoursePurchase` avec `amount_paid` mais ne déduit ni `dt_balance` ni `token_balance`. C'est un enregistrement comptable sans mouvement de fonds réel. Seul l'achat de pack (`packs.py:162`) déduit réellement du `dt_balance`.

4. Suivi de progression dans une formation

✗ ABSENT

Vérification	Résultat	Preuve
Suivi de progression formation enseignant	Inexistant	Le modèle <code>Progress/LessonProgress</code> existe pour les cours étudiants mais pas pour les formations enseignant (pas de modèle dédié)
Certificat de complétion	Générique uniquement	Le modèle <code>Certificate</code> (<code>models.py:1301-1323</code>) est générique (tout rôle) mais il est lié aux <code>CourseEnrollment</code> — pas de mécanisme pour les formations enseignant qui n'existent pas

PARTIE 2 — PARCOURS ÉLÈVE : PARCOURS D'APPRENTISSAGE PERSONNALISÉ

5. Détermination du niveau scolaire

⚠ PARTIEL

Vérification	Résultat	Preuve
Déclaration à l'inscription	Fonctionne	RegisterPage.tsx:219-234 — dropdown avec niveaux tunisiens (Primaire/Préparatoire/Secondaire). auth.py:108 — niveau_scolaire sur User
Modèle de données	Fonctionne	User.niveau_scolaire (models.py:296) — String(50), nullable. Enum NiveauScolaire (models.py:157-191) définit les valeurs valides
Import CSV admin	Fonctionne	admin.py:2576,2604,2626 — CSV format email, full_name, password, niveau_scolaire
Modification après inscription	✗ ABSENT	users.py:109-142 — update_user() ne gère que full_name, is_active Pas de mise à jour de niveau_scolaire possible
Conflit niveau élève vs classe	✗ ABSENT	Pas de modèle Classroom qui assignent un niveau à une classe champ Classroom (models.py:114-133) a un champ level mais aucune logique de cohérence avec User.niveau_scolaire

Conclusion : Le niveau est défini une fois à l'inscription et n'est pas modifiable ensuite — ni par l'élève, ni par l'admin, ni par l'enseignant.

6. Évaluation du niveau de compétence

✗ ABSENT

Vérification	Résultat	Preuve
Quiz de positionnement	Inexistant	Aucun mécanisme diagnostic, placement, assessment, competency, les modèles ou routers

Vérification	Résultat	Preuve
Quiz existants	Uniquement cours-internes	Le système de quiz (Quiz, QuizQuestion, QuizAttempt) est attaché au cours (learner.py:494-664). Pas de quiz standalone de diagnostic
Impact sur recommandation	Inexistant	Pas de système de recommandation

7. Recommandation de parcours d'apprentissage

✗ ABSENT

Vérification	Résultat	Preuve
Moteur de recommandation	Inexistant	Aucun endpoint/service avec recommandation, learning_path, parcours, recommandation
Filtrage par niveau scolaire dans le catalogue	✗ ABSENT	catalog.py:15-69 — filtre par category, level (beginner/intermediate/advanced), language de filtre niveau_scolaire. L'élève de 6ème primaire voit les mêmes recommandations que celui de 4ème année
Page "parcours d'apprentissage"	Inexistant	Aucune route /student/learning-path ou similaire dans App.tsx
Recommandation UI	Inexistant	Aucun composant "suggested", "for you", "recommended" dans le UI

8. Découverte des packs disponibles

⚠ PARTIEL

Vérification	Résultat	Preuve
Filtrage par niveau scolaire	Backend OK	packs.py:39-40 — GET /api/packs supporte ?niveau_scolaire=...
Page frontend packs pour élèves	✗ ABSENT	Aucune route /student/packs dans App.tsx. Le wallet étudiant (StudentWallet.tsx) affiche les soldes mais pas de catalogue de packs
Distinction eduai_catalog vs independent_teacher	✗ ABSENT	StudyPack (models.py:723-759) n'a pas de champ owner_type. Le Course a un owner_type mais le pack non. Aucune distinction visuelle possible
"Déjà inclus dans votre école"	✗ ABSENT	Le listing packs (packs.py:29-61) ne vérifie pas si l'école de l'élève a déjà un pack actif pour ce niveau
Achat pack individuel vs école	Backend OK, frontend absent	L'achat individuel (packs.py:101-203) vérifie si l'étudiant a déjà un pack individuel pour ce niveau (lignes 138-149) mais ne vérifie pas le pack école

9. Parcours d'achat d'un cours ou pack (côté élève)

⚠ PARTIEL — Fonctionnellement cassé

Achat de cours à l'unité :

Étape	Résultat	Preuve
Prix affiché	Fonctionne	CourseCatalog.tsx:252 — bouton "Acheter" visible si !isEnrolled && !isFree
Déclenchement paiement	Fonctionnellement cassé	courses.py:309-366 — crée un CoursePurchase sans déduire aucun

Étape	Résultat	Preuve
		solde et sans appeler Stripe. L'élève "achète" gratuitement
Débit réel	✗ NE DÉbite PAS	Aucun appel à <code>dt_balance</code> , <code>token_balance</code> , ou Stripe dans <code>purchase_course</code>
Accès post-achat	Fonctionne (par accident)	Ligne 352 crée un <code>CourseEnrollment</code> — l'accès est accordé même sans paiement
Cas échec paiement	Inexistant	Pas de vérification de solde = pas d'erreur possible

Achat de pack individuel :

Étape	Résultat	Preuve
Prix affiché	Backend OK	<code>packs.py:48-59</code> retourne <code>price</code> , <code>currency</code>
Déclenchement paiement	Fonctionne	<code>packs.py:101-203</code> — déduit de <code>user.dt_balance</code> (ligne 162) et crée un <code>Transaction</code>
Vérification solde	Fonctionne	<code>packs.py:155-158</code> — vérifie <code>user.dt_balance >= study_pack.price</code>
Accès post-achat	Fonctionne	<code>course_access.py:58-69</code> — vérifie <code>PackPurchase</code> actif + correspondance <code>niveau_scolaire</code>
Nouveau cours ajouté après achat	Fonctionne	L'accès est vérifié dynamiquement par <code>has_course_access()</code> — tout cours du même niveau couvert par le pack est accessible

MAIS : `has_course_access()` n'est JAMAIS APPELÉ par les routers. Voir point 11.

10. Cohérence accès école vs achat individuel

⚠ PARTIEL

Vérification	Résultat	Preuve
Double achat inutile (école + individuel)	✗ ABSENT	packs.py:138-149 vérifie le pack individuel de l'étudiant mais pas le pack école. Un élève peut acheter un pack individuel même si son école en a déjà un
Accès via pack école	Backend OK, non utilisé	course_access.py:72-83 vérifie le pack école. MAIS cette fonction n'est jamais appelée par les routers — voir point 11
Accès via pack individuel	Backend OK, non utilisé	course_access.py:58-69 — même constat
Accès cours gratuit	Fonctionne	learner.py:274-302 — inscription gratuite directe, pas de vérification de pack
Test des 3 cas (école/individuel/gratuit)	Impossible	Le système d'accès intégré (has_course_access) n'est pas câblé

PARTIE 3 — TESTS TRANSVERSAUX DE SÉCURITÉ

11. Fuite de contenu non autorisé dans le catalogue

⚠ PARTIEL

Vérification	Résultat	Preuve
Cours draft dans catalogue public	✓ PROTÉGÉ	catalog.py:27-30 — filtre status PUBLISHED exclut les drafts
Cours pending_review dans catalogue	✓ PROTÉGÉ	Même filtre status == PUBLISHED exclut pending_re

Vérification	Résultat	Preuve
Cours visibility=private dans catalogue	✓ PROTÉGÉ	catalog.py:30 — filtre visibility = "public_catalog"
Cours visibility=school_only dans catalogue général	✓ PROTÉGÉ	Même filtre
Cours school_only accessible par slug	✗ FAILLE	catalog.py:75-77 — GET /catalog/courses/{slug} filtre par status == PUBLISHED mais pas par visibility. Un cours school_only publié est accessible par slug depuis le catalogue public
Cours school_only dans /api/learner/courses/{id}	⚠ PARTIEL	learner.py:203-206 — filtre is_published == True mais pas visibility
Catégories exposant du contenu non-public	⚠ PARTIEL	catalog.py:127-129 — /catalog/categories filtre par status == PUBLISHED mais pas visibility

12. Fuite cross-école dans les recommandations/catalogue

⚠ PARTIEL

Vérification	Résultat	Preuve
Tenant filter ORM sur catalogue public	✗ NON APPLICABLE	Le tenant filter (_add_tenant_filter dans db/session.py) filtre les modèles avec school_id. Le catalog.py utilise une requête ORM classique → le filtre s'applique. Les

Vérification	Résultat	Preuve
		cours public_catalog n'ont pas de school_id filtrant
Cours school_only d'une autre école dans le catalogue	✓ PROTÉGÉ	Le filtre visibility == "public_catalog" dans catalog.py:30 exclut les cours school_only du catalogue public. Le tenant filter s'applique sur les requêtes ORM scolaires
Cours school_only d'une autre école via /api/learner/courses/{id}	⚠ RISQUE	learner.py:203-206 — pas de filtre visibility. Si un cours school_only a is_published=True, un élève d'une autre école pourrait y accéder directement par ID. Le tenant filter devrait bloquer mais c'est une dépendance implicite
Catalogue académique (/api/academy/courses)	✓ PROTÉGÉ	academy.py:42-57 — filtre par Course.school_id == user.school_id pour les étudiants

13. Cohérence du prix affiché vs prix réellement débité

✗ FAILLE CRITIQUE

Vérification	Résultat	Preuve
Prix affiché dans le catalogue	Fonctionne	CourseCatalog.tsx:252 — affiche le bouton "Acheter" pour les cours payants
Prix réellement	✗ RIEN N'EST DÉBITÉ	courses.py:309-366 — purchase_course ne déduit aucun solde. Crée

Vérification	Résultat	Preuve
débité (cours)		un CoursePurchase avec amount_paid=course.price mais l'argent ne bouge pas
Prix affiché vs débité (pack individuel)	 COHÉRENT	packs.py:155-162 — vérifie user.dt_balance >= study_pack.price puis déduit study_pack.price
Prix affiché vs débité (pack école)	 COHÉRENT	admin.py:2363-2371 — vérifie admin.dt_balance >= pack.price puis déduit

RÉSUMÉ DES STATUTS

#	Point testé	Statut
1	Catalogue de formations enseignant	 ABSENT
2	Restriction par état enseignant	 ABSENT
3	Parcours d'achat formation	 ABSENT
4	Suivi progression formation	 ABSENT
5	Détermination niveau scolaire	 PARTIEL — défini à l'inscription, non modifiable
6	Évaluation compétence	 ABSENT

#	Point testé	Statut
7	Recommandation parcours	✗ ABSENT
8	Découverte packs	⚠ PARTIEL — backend OK, frontend absent
9	Achat cours/pack	⚠ PARTIEL — cours: ne débite pas; packs: OK
10	Cohérence école vs individuel	⚠ PARTIEL — double achat possible
11	Fuite contenu non autorisé	⚠ PARTIEL — slug expose school_only
12	Fuite cross-école	⚠ PARTIEL — tenant filter dépendant
13	Prix affiché vs débité	✗ FAILLE — cours: rien n'est débité

TOP 3 LACUNES PRIORITAIRES

1. ● has_course_access() n'est jamais appelé (bloquant)

Fichier : backend/app/services/course_access.py:20-85 — fonction définie, jamais importée ni appelée par aucun router.

Impact : Toute la logique d'accès par pack (individuel OU école) est mort-née. Les vérifications de pack, de niveau scolaire, d'expiration — tout est codé mais inutilisé. Les cours sont accessibles à quiconque s'inscrit (gratuitement ou via purchase_course qui ne débite rien).

Correccion requise : Câbler has_course_access() comme dependency ou middleware sur les endpoints d'accès aux cours (learner.py, courses.py).

2. ● purchase_course ne débite aucun solde (faille financière)

Fichier : backend/app/routers/courses.py:309-366

Impact : Tout cours payant peut être "acheté" gratuitement. Le endpoint crée un CoursePurchase + CourseEnrollment sans vérifier ni débit de dt_balance, token_balance, ni appel à Stripe. L'élève obtient l'accès sans payer.

Correccion requise : Ajouter la vérification de solde + déduction, ou câbler Stripe Checkout comme pour les packs.

3. ● Parcours enseignant : coquille UI sans backend (fonctionnalité fantôme)

Fichier : frontend/src/features/teacher/pages/MyLearning.tsx —
route /dashboard/teacher/learning active, UI complète, tests mockés.

Impact : L'enseignant accède à une page qui affiche "Aucune formation disponible" ou plante (appels vers des endpoints inexistants). Aucun modèle, aucun endpoint, aucune logique métier ne supporte cette fonctionnalité.

Correccion requise : Soit implémenter le backend complet (modèle Training, endpoints CRUD, catalogue, achats), soit retirer la route du frontend pour éviter la confusion.

Note sécurité additionnelle — la faille GET /catalog/courses/{slug} (catalog.py:75-77) qui expose les cours school_only par slug mérite une correction immédiate (ajouter Course.visibility == "public_catalog" au filtre). C'est un fix d'une ligne.